K8S-08: DQL Queries for Kubernetes

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 8 of 13 | **Created:** January 2026 | **Last Updated:** 07/30/2026

Advanced Query Patterns for Kubernetes Data

This notebook provides a comprehensive reference of DQL queries for Kubernetes monitoring. From basic entity queries to complex performance analysis, these patterns help you extract insights from your Kubernetes data.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring

Requirement	Details
Permissions	`metrics.read`, `entities.read`, `logs.read`
Knowledge	K8S-01 through K8S-07
Data	Active Kubernetes cluster monitored

1. Entity Queries

Kubernetes Entity Types

Entity Type	DQL Table	Description
Cluster	`dt.entity.kubernetes_cluster`	K8s clusters
Node	`dt.entity.kubernetes_node`	Worker nodes
Namespace	`dt.entity.cloud_application_namespace`	Namespaces
Workload	`dt.entity.cloud_application`	Deployments, StatefulSets
Container	`dt.entity.container_group_instance`	Container instances
Service	`dt.entity.service`	Detected services

```
// List all Kubernetes clusters (smartscape topology)
smartscapeNodes "K8S_CLUSTER"
| fields entity.name = name, tags
| sort entity.name asc

// Legacy alternative (deprecated for new content):
// fetch dt.entity.kubernetes_cluster
// | fields entity.name, tags
// | sort entity.name asc
```

```
// List all Kubernetes nodes (smartscape topology)
smartscapeNodes "K8S_NODE"
| fields entity.name = name, tags
| sort entity.name asc

// Legacy alternative (deprecated for new content):
// fetch dt.entity.kubernetes_node
// | fields entity.name, tags
// | sort entity.name asc
```

```
// List all namespaces (smartscape topology)
smartscapeNodes "K8S_NAMESPACE"
| fields entity.name = name, tags
| sort entity.name asc

// Legacy alternative (deprecated for new content):
// fetch dt.entity.cloud_application_namespace
// | fields entity.name, tags
// | sort entity.name asc
```

```
// List all workloads — Deployments (smartscape topology)
smartscapeNodes "K8S_DEPLOYMENT"
| fields entity.name = name, tags
| sort entity.name asc
| limit 50

// For other workload types, substitute:
// smartscapeNodes "K8S_STATEFULSET"
// smartscapeNodes "K8S_DAEMONSET"

// Legacy alternative (deprecated for new content):
// fetch dt.entity.cloud_application
// | fields entity.name, tags
// | sort entity.name asc
// | limit 50
```

```
// Count Kubernetes nodes (smartscape topology)
smartscapeNodes "K8S_NODE"
| summarize nodeCount = count()
```

2. Metric Queries

Common Kubernetes Metrics

Metric	Description	Unit
`dt.kubernetes.container.cpu_usage`	Container CPU usage	Millicores
`dt.kubernetes.container.memory_working_set`	Container memory (active)	Bytes
`dt.kubernetes.container.requests_cpu`	CPU requests	Millicores
`dt.kubernetes.container.requests_memory`	Memory requests	Bytes
`dt.kubernetes.container.limits_cpu`	CPU limits	Millicores
`dt.kubernetes.container.limits_memory`	Memory limits	Bytes
`dt.containers.cpu.throttled_time`	CPU throttle time	Nanoseconds
`dt.kubernetes.node.cpu_usage`	Node CPU usage	Millicores
`dt.kubernetes.node.memory_usage`	Node memory usage	Bytes

> **Reading request and limit metrics across an ActiveGate 1.343 upgrade. ActiveGate 1.343 changes what the CPU and memory *request* and *limit* metrics count: init containers are now included in the pod-scope total** — and therefore in any workload- or namespace-level roll-up of it. Reported reservations **step up at the upgrade with no workload change at all**, because the arithmetic changed, not the cluster.
>
> | Metric family | Affected by the 1.343 change? |
> |---|---|
> | `*.requests_cpu`, `*.requests_memory`, `*.limits_cpu`, `*.limits_memory` | **Yes** — init containers now counted |
> | `*.cpu_usage`, `*.memory_working_set`, `*.memory_usage` | **No** — usage metrics are unchanged |
>
> Two practical consequences:

1. **A trend that spans the upgrade boundary is two series, not one.** Do not read a request/limit chart across the boundary as a single trend, and do not let a threshold alert on reserved CPU or memory fire on the discontinuity. Bound your comparison windows on one side of the upgrade or the other.
2. **Pre-1.343 baselines understate reservations** relative to what the scheduler actually reserved, because init-container requests were excluded. Post-1.343 figures are the more faithful number — re-baseline rather than trying to reconcile the two.

ActiveGate 1.343 rolls out per ActiveGate, not per tenant, and ActiveGate fleets lag tenant version — so **verify the version of the ActiveGate carrying the `kubernetes-monitoring` capability for the cluster in question** before deciding which side of the boundary a data point sits on. Until 1.343 reaches that ActiveGate, the pre-1.343 reading (init containers excluded) is what your data shows, and everything above about baselines and alert thresholds still describes it correctly.

```
// Container CPU usage - top consumers
timeseries avgCpuMillicores = avg(dt.kubernetes.container.cpu_usage),
from:-1h, by:{dt.entity.container_group_instance}
| sort avgCpuMillicores desc
| limit 15
```

```
// Container memory usage approaching limits
timeseries avgMemBytes = avg(dt.kubernetes.container.memory_working_set),
from:-1h, by:{dt.entity.container_group_instance}
| fieldsAdd avgMemBytesValue = arrayAvg(avgMemBytes)
| sort avgMemBytesValue desc
```

```
// CPU throttling detection
timeseries totalThrottle = sum(dt.containers.cpu.throttled_time), from:-1h,
by:{dt.entity.container_group_instance}
| fieldsAdd totalThrottleValue = arrayAvg(totalThrottle)
| filter totalThrottleValue &gt; 0
| sort totalThrottleValue desc
| limit 15
```

```
// Namespace-level resource usage
timeseries avgCpuUsageMillicores = avg(dt.kubernetes.container.cpu_usage),
from:-1h, by:{k8s.namespace.name}
| sort avgCpuUsageMillicores desc
| limit 15
```

```
// Resource requests by namespace (capacity planning)
timeseries avgCpuRequests = avg(dt.kubernetes.workload.requests_cpu),
from:-1h, by:{k8s.namespace.name}
| sort avgCpuRequests desc
| limit 15
```

3. Log and Event Queries

Log Query Patterns

```
// Error logs by namespace
fetch logs, from:-1h
| filter loglevel == "ERROR"
| filter isNotNull(k8s.namespace.name)
| summarize errorCount = count(), by:{k8s.namespace.name}
| sort errorCount desc
| limit 15
```

```
// Kubernetes events - warnings only
fetch logs, from:-1h
| filter matchesPhrase(content, "Warning")
| filter matchesPhrase(log.source, "kubernetes") or matchesPhrase(log.source,
"k8s")
| fields timestamp, content
| sort timestamp desc
| limit 30
```

```
// Pod crashes and restarts
fetch logs, from:-1h
| filter matchesPhrase(content, "CrashLoopBackOff") or matchesPhrase(content,
"BackOff")
| fields timestamp, content
| sort timestamp desc
| limit 20
```

```
// OOM events
fetch logs, from:-1h
| filter matchesPhrase(content, "OOMKilled") or matchesPhrase(content, "Out of memory")
| fields timestamp, content
| sort timestamp desc
| limit 20
```

```
// Log volume trend by namespace
fetch logs, from: now() - 24h
| filter isNotNull(k8s.namespace.name)
| summarize log_count = count(), by:{k8s.namespace.name, time_bucket = bin(timestamp, 1h)}
| sort time_bucket asc
| limit 200
```

4. Trace Queries

Span Queries for Kubernetes Services

```
// Service response times in K8s
fetch spans, from:-1h
| filter span.kind == "server"
| filter isNotNull(k8s.namespace.name)
| summarize
    p50 = percentile(duration, 50),
    p95 = percentile(duration, 95),
    p99 = percentile(duration, 99),
    by:{k8s.namespace.name, dt.entity.service}
| sort p99 desc
| limit 15
```

```
// Error rate by service
fetch spans, from:-1h
| filter span.kind == "server"
| filter isNotNull(k8s.namespace.name)
| summarize
    total = count(),
    errors = countIf(otel.status_code == "ERROR"),
    by:{k8s.namespace.name, dt.entity.service}
| fieldsAdd errorRate = 100.0 * toDouble(errors) / toDouble(total)
| filter errors &gt; 0
| sort errorRate desc
| limit 15
```

```
// Slow traces (&gt;1 second)
fetch spans, from:-1h
| filter span.kind == "server" and duration &gt; 1000000000
| filter isNotNull(k8s.namespace.name)
| fields timestamp, trace.id, k8s.namespace.name, span.name, duration
| sort duration desc
| limit 20
```

```
// Request throughput by namespace
fetch spans, from:-1h
| filter span.kind == "server"
| filter isNotNull(k8s.namespace.name)
| summarize requestCount = count(), by:{k8s.namespace.name}
| sort requestCount desc
| limit 10
```

5. Correlation Queries

Joining Multiple Data Sources

```
// High CPU workloads with their names
timeseries avgCpuMillicores = avg(dt.kubernetes.container.cpu_usage),
from:-1h, by:{dt.entity.cloud_application}
| fieldsAdd avgCpuMillicoresValue = arrayAvg(avgCpuMillicores)
| lookup [fetch dt.entity.cloud_application | fields id, entity.name],
sourceField:dt.entity.cloud_application, lookupField:id
| fieldsRename workloadName = lookup.entity.name
| fields workloadName, avgCpuMillicoresValue
| sort avgCpuMillicoresValue desc
```

```
// Nodes with high utilization
timeseries avgNodeCpu = avg(dt.kubernetes.node.cpu_usage), from:-1h, by:
{dt.entity.kubernetes_node}
| fieldsAdd avgNodeCpuValue = arrayAvg(avgNodeCpu)
| filter avgNodeCpuValue &gt; 70
| lookup [fetch dt.entity.kubernetes_node | fields id, entity.name],
sourceField:dt.entity.kubernetes_node, lookupField:id
| fieldsRename nodeName = lookup.entity.name
| fields nodeName, avgNodeCpuValue
| sort avgNodeCpuValue desc
```

6. Dashboard Queries

Queries Optimized for Dashboards

```
// Cluster summary tile (smartscape topology)
smartscapeNodes "K8S_CLUSTER"
| summarize clusterCount = count()
```

```
// Node count tile (smartscape topology)
smartscapeNodes "K8S_NODE"
| summarize nodeCount = count()
```

```
// Error rate trend (chart)
fetch logs, from: now() - 24h
| filter loglevel == "ERROR" and isNotNull(k8s.namespace.name)
| summarize errors = count(), by:{time_bucket = bin(timestamp, 1h)}
| sort time_bucket asc
```

```
// Top namespaces by CPU (bar chart)
timeseries avgCpuMillicores = avg(dt.kubernetes.container.cpu_usage),
from:-1h, by:{k8s.namespace.name}
| sort avgCpuMillicores desc
| limit 10
```

7. Alerting Queries

Threshold-Based Alerting Patterns

```
// High memory usage alert query
timeseries avgMemBytes = avg(dt.kubernetes.container.memory_working_set),
from:-1h, by:{dt.entity.container_group_instance}
| fieldsAdd avgMemBytesValue = arrayAvg(avgMemBytes)
```

```
// CrashLoop detection (last hour)
fetch logs, from: now() - 1h
| filter matchesPhrase(content, "CrashLoopBackOff")
| summarize crashCount = count()
```

```
// Error rate spike detection
fetch logs, from: now() - 1h
| filter loglevel == "ERROR" and isNotNull(k8s.namespace.name)
| summarize errors = count(), by:{k8s.namespace.name}
| filter errors &gt; 100
| sort errors desc
```

8. Query Templates

Reusable Query Patterns

Filter by Namespace:

```
| filter k8s.namespace.name == "NAMESPACE_NAME"
```

Filter by Cluster:

```
| filter kubernetesClusterName == "CLUSTER_NAME"
```

Time-based Analysis:

```
fetch logs, from: now() - DURATION
| summarize log_count = count(), by:{time_bucket = bin(timestamp, INTERVAL)}
| sort time_bucket asc
```

Top N Pattern:

```
| sort METRIC desc
| limit N
```

Summary

This notebook provided DQL query patterns for:

- Entity discovery and relationships
- Container and node metrics
- Log and event analysis
- Distributed tracing
- Multi-source correlation
- Dashboard visualizations
- Alerting conditions

References

- Dynatrace Query Language (DQL) reference (DT docs)
- smartscapeNodes command (DT docs)
- Set up Dynatrace on Kubernetes (DT docs)
- Kubernetes app — workloads + namespaces views (DT docs)
- Davis Problems app (DT docs)
